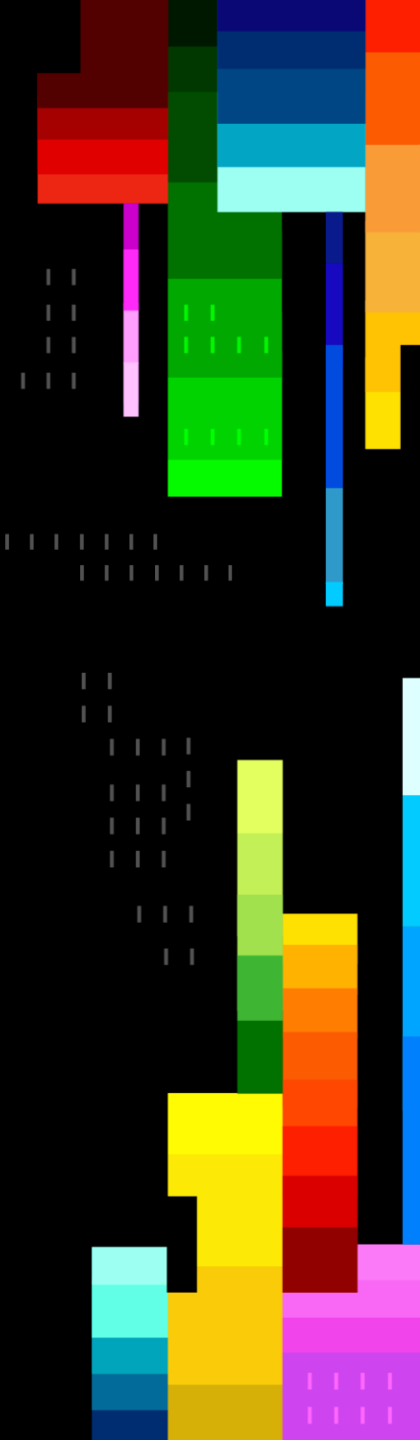
Microsoft

LAB520: Train the Trainer

Get Started with Models in Microsoft Foundry
Microsoft Build 2026



About this session

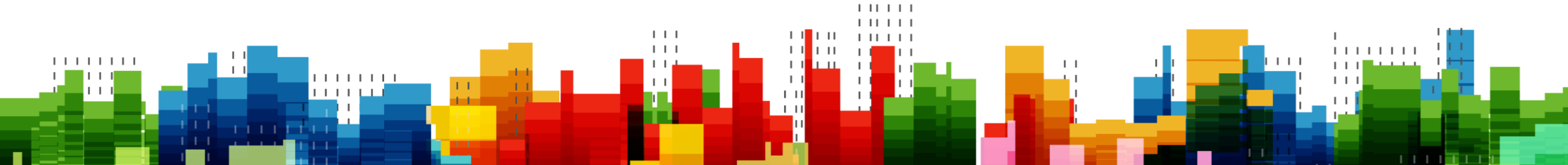
A Level 300 hands-on lab teaching developers to go from model discovery to a deployed hosted agent in Microsoft Foundry.

- Attendees play Serena, a developer at Zava (a global home-improvement retailer), and build a product review moderation pipeline — discovery to hosted agent, in Python, no fine-tuning required.
  - Source repo: github.com/microsoft/Build26-LAB520 • Duration: 75 minutes • Level 300

Agenda

- 1. Lab overview and audience
- 2. Learning objectives and prerequisites
- 3. Lab flow: 7 labs from model discovery to hosted agent
- 4. Lab-by-lab teaching notes
- 5. Common pitfalls and timing guidance
- 6. Demos, talking points, and resources

About the Lab



Audience & Outcomes

Who should attend

- App developers and cloud engineers building AI features
- Comfortable with Python and REST APIs
- Familiar with Azure fundamentals (subscription, resource group)
- No prior ML or fine-tuning experience required

What they will leave with

- Working Python code connecting to a Foundry-hosted model
- A product review moderation pipeline for Zava in Python
- A hosted agent deployed to their Foundry project
- Confidence comparing models in the Foundry catalog

Learning objectives

- Navigate the Microsoft Foundry model catalog and select a model by capability and cost
- Provision a Foundry project with azd + Bicep and configure identity-based auth
- Use the Azure AI Projects SDK (AIProjectClient + OpenAI-compatible client) for a first inference
- Build Zava's product review moderation prompt with structured JSON output and confidence-based business logic
- Compare outputs across models and justify a model choice
- Package the workflow as a hosted agent with the Microsoft Agent Framework and deploy via azd up

Prerequisites

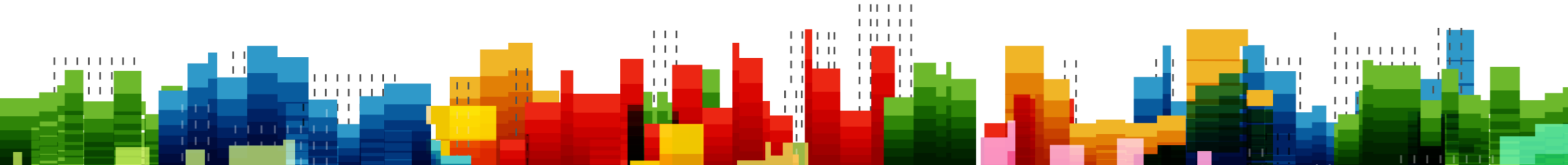
For attendees

- Azure subscription with rights to create Foundry resources
- Python 3.10+, VS Code, and Git installed
- Azure CLI + Azure Developer CLI (azd) installed; 'az login' and 'azd auth login' working
- Clone of https://github.com/microsoft/Build26-LAB520 (contains azure.yaml, infra/ Bicep, scripts/setup)

For trainers

- Run the lab end-to-end at least once before delivery
- Pre-provision a backup Foundry project for attendees who hit quota
- Know your room's network — model calls need outbound HTTPS
- Have model deployment names and region ready to share on a slide

Facilitating the Lab



Lab flow at a glance

Discover & Provision (Labs 1–2)

- Browse the Foundry model catalog at ai.azure.com
- Provision project + model with azd + Bicep
- Deploy gpt-4.1-mini (and optional gpt-4.1)
- Auto-write .env and validate with validate_lab.py

Inference & Moderation (Labs 3–5)

- Connect with AIProjectClient + DefaultAzureCredential
- Run first inference (Lab 3: 01_first_inference.py)
- Build Zava review moderation pipeline (Lab 4)
- Compare gpt-4.1-mini vs gpt-4.1 (Lab 5, optional)

Hosted Agent (Labs 6–7)

- Package with Microsoft Agent Framework + Dockerfile
- Deploy to Foundry Agent Service with azd up
- Test with azd ai agent invoke + Foundry Playground
- Wrap up + cleanup with azd down (Lab 7)

Labs 1 & 2 — Discover and Provision

What attendees do

- Lab 1: Open ai.azure.com, ensure new Foundry toggle on, browse the catalog
- Filter by chat completion + try the Zava moderation prompt in Playground
- Lab 2: Run setup.ps1 / setup.sh — azd provisions project, model, RBAC, .env
- Verify in Foundry portal + run validate_lab.py (100 checks pass)

Teaching notes

- Recommend gpt-4.1-mini — cheapest, fast, ideal for moderation
- Set -DeploySecondModel to also deploy gpt-4.1 for Lab 5
- Provisioning takes 5–10 min — keep attendees engaged with the Playground
- Default region northcentralus; swedencentral / eastus2 as fallbacks

Lab 3 — Connect and Send Your First Inference

What attendees do

- Use AIProjectClient (azure-ai-projects) + DefaultAzureCredential
- Get OpenAI-compatible client via project_client.get_openai_client()
- Run src/01_first_inference.py against gpt-4.1-mini
- Inspect choices, usage, finish_reason — make tokens and cost concrete

Teaching notes

- Emphasise identity-based auth over API keys
- Show the response object — many learners skip past 'usage'
- If calls fail: check .env endpoint, MODEL_DEPLOYMENT_NAME, az login scope
- Print tokens_in / tokens_out to make cost real

Lab 4 — Zava Review Moderation Pipeline

What attendees do

- Write a system prompt that returns JSON: SAFE / NEEDS_REVIEW / UNSAFE + confidence + reason
- Implement classify_comment() with temperature=0.0 for determinism
- Add apply_moderation business logic: APPROVED / BLOCKED / FLAGGED_FOR_REVIEW
- Run batch + interactive modes via src/02_comment_moderation.py

Teaching notes

- Frame as Zava's review pipeline (Bruno's kitchen renovation reviews)
- Thresholds: SAFE ≥0.8 auto-approve; UNSAFE ≥0.7 block; else flag
- Model gives the label; your code makes the decision — that is the takeaway
- Lab 5 (optional, self-paced): compare gpt-4.1-mini vs gpt-4.1, --hybrid mode

Labs 6 & 7 — Deploy a Hosted Agent and Wrap Up

What attendees do

- Build agent in src/agent/app.py with FoundryChatClient + ResponsesHostServer
- Add Dockerfile + agent.yaml manifest; smoke-test locally with python app.py
- Deploy with azd up — builds image, pushes to ACR, registers the agent
- Invoke via azd ai agent invoke + Foundry Playground; Lab 7 wrap-up + azd down

Teaching notes

- Microsoft Agent Framework wraps the model with the OpenAI Responses API
- azd up is the magic moment — show the live build, push, and registration logs
- Use azd ai agent monitor to tail traces; show it in the Foundry portal too
- End with azd down — clean up so attendees don't burn quota after the session

Common pitfalls (and how to unstick attendees)

Setup & auth

- azd auth login + az login both required — DefaultAzureCredential needs az
- azd provision quota errors — switch region to swedencentral or eastus2
- .env missing or stale — re-run setup.ps1/setup.sh, then validate_lab.py

SDK & inference

- Deployment name in .env must match the Foundry deployment exactly (case-sensitive)
- JSON parse fails — model returned prose; remind learners to set response_format
- First call slow — Foundry cold start; not an error, just wait it out

Agent deploy

- ACR / capability host not enabled — azd up surfaces this; rerun after the fix
- agent.yaml drift from .env — model name, endpoint, or version don't match
- Forgot azd down — costs keep running; remind everyone at the end of Lab 7

Timing & pacing (75 minutes)

- 0–5 min — Welcome, prerequisites check, login verification
- 5–20 min — Labs 1 & 2: Discover models + azd provision (run setup, talk over the wait)
- 20–35 min — Lab 3: First inference with AIProjectClient (01_first_inference.py)
- 35–55 min — Lab 4: Build Zava's review moderation pipeline (Lab 5 optional, self-paced)
- 55–70 min — Lab 6: Deploy hosted agent with azd up + invoke / monitor
- 70–75 min — Lab 7: Wrap-up, azd down cleanup, Q&A, next steps

Key demos & talking points

- Open ai.azure.com — filter by chat-completion to find gpt-4.1-mini
- Run azd up live — project, model, RBAC, .env all in one command
- Lab 4 interactive: feed Bruno's "garbage paint" review → UNSAFE → BLOCKED
- Show a NEEDS_REVIEW case — model labels, your code decides (the takeaway)
- End with azd ai agent invoke — same logic, now a hosted endpoint

Resources & support

For trainers

- Lab repo: https://github.com/microsoft/Build26-LAB520
- Foundry docs: https://learn.microsoft.com/azure/ai-foundry
- Foundry Discord: https://aka.ms/foundry/discord
- Model catalog: https://ai.azure.com

Hand off to attendees

- Lab walkthroughs: Build26-LAB520/docs (lab0–lab7 markdown guides)
- Code samples in the lab repo
- Microsoft Foundry DevBlogs: devblogs.microsoft.com/foundry
- Follow up: send attendees the session feedback form